Systems Engineering of LLM Workstation

Kirill Nevzorov under supervision
of Prof. Christopher Buckley
University of Sussex
School of Engineering and Informatics

March 4, 2026

Abstract

We wish to research the process of creating and use-case testing various LLM prompts, such autonomously prompting the user for additional details relevant to their project, assisting them in and not limited to: re-sumarising and verifying the information stored within a workspace, and generating intelligence from a notes archive, and discussing progress and completion stages. We research and employ the mechanism of configuring an LLM workstation and using it effectively.

Contents

I. Introduction

1.1 Background and Motivation

To build useful software it is important to understand the software and its purpose as its being built. Domain knowledge is not always accessible when a new software is being built, and it must be gathered continuously as the codebase evolves.

In order to build a software that will help the users with creating intelligence around a code base, a project, or otherwise a collection of files, we need to be able to interface with an LLM, a large language model, which will be the backbone of analysing the contents of the repository.

1.2 Project Scope and Objectives

We wish to interact with Large Language Models efficiently...

1.3 Report Structure

This report is organised into six main chapters, followed by appendices:

Chapter I: Introduction provides the background and motivation for this project, defines the scope and objectives, and outlines the structure of the report.

Chapter II: Foundations of LLM Systems establishes the theoretical groundwork by examining large language model architectures, distinguishing between inference and training workloads, characterising LLM workload requirements in terms of latency, throughput, and resource utilisation, and identifying key system integration challenges.

Chapter III: Architectural Patterns and Design explores the design space for LLM serving systems, covering inference serving architectures (single model, multi-model ensembles, and model routing), backend selection strategies (hardware, software stack, and cost-benefit considerations), query management and optimisation techniques (queuing, caching, and batch processing), and scalability patterns.

Chapter IV: Practical Implementation Considerations addresses the engineering aspects of deploying LLM systems, including infrastructure and deployment (containerisation, cloud vs. on-premise, resource provisioning), monitoring and observability (metrics, error handling, logging), and security and access control.

Chapter V: Research Findings and Analysis presents the results of our exploration, including evaluations of LLM routers and load balancers (with specific focus on the llama.cpp ecosystem and Ollama architecture), system design tradeoffs (cost vs. performance,

complexity vs. maintainability, flexibility vs. stability), and lessons learned.

Chapter VI: Conclusion and Future Work summarises the key findings, provides recommendations for implementation, and identifies areas for further research.

Appendices include code examples and snippets, system diagrams and architecture charts, and research resources and references.

II. Foundations of LLM Systems

2.1 Understanding Large Language Models

2.1.1 Model Architecture Overview

2.1.2 Inference vs. Training Workloads

2.2 LLM Workload Characteristics

2.2.1 Latency Requirements

2.2.2 Throughput Considerations

2.2.3 Resource Utilization Patterns

2.3 System Integration Challenges

III. Architectural Patterns and Design

3.1 LLM Inference Serving Architectures

3.1.1 Single Model Serving

3.1.2 Multi-Model Ensembles

3.1.3 Model Routing and Load Balancing

3.2 Backend Selection Strategies

3.2.1 Hardware Considerations

3.2.2 Software Stack Evaluation

3.2.3 Cost-Benefit Analysis

3.3 Query Management and Optimization

3.3.1 Request Queuing and Prioritization

3.3.2 Caching Strategies

3.3.3 Batch Processing Approaches

3.4 Scalability and Performance Patterns

IV. Practical Implementation Considerations

4.1 Infrastructure and Deployment

4.1.1 Containerization and Orchestration

4.1.2 Cloud vs. On-Premise Considerations

4.1.3 Resource Provisioning

4.2 Monitoring and Observability

4.2.1 Performance Metrics

4.2.2 Error Handling and Recovery

4.2.3 Logging and Debugging

4.3 Security and Access Control

Appendices

A. Code Examples and Snippets

B. System Diagrams and Architecture Charts

C. Research Resources and References